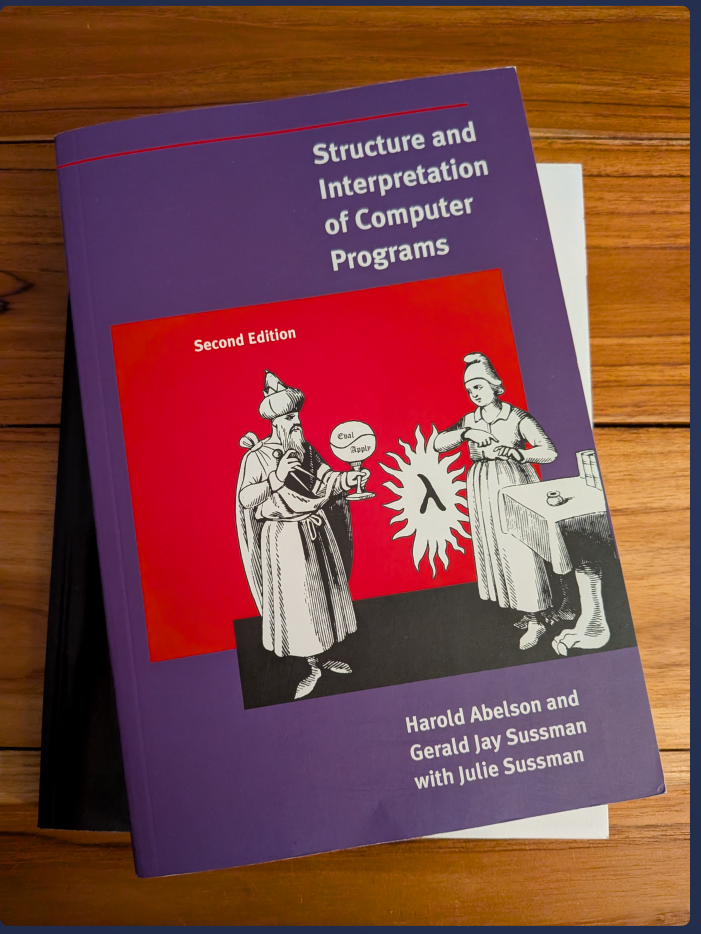
API Design is Language Design

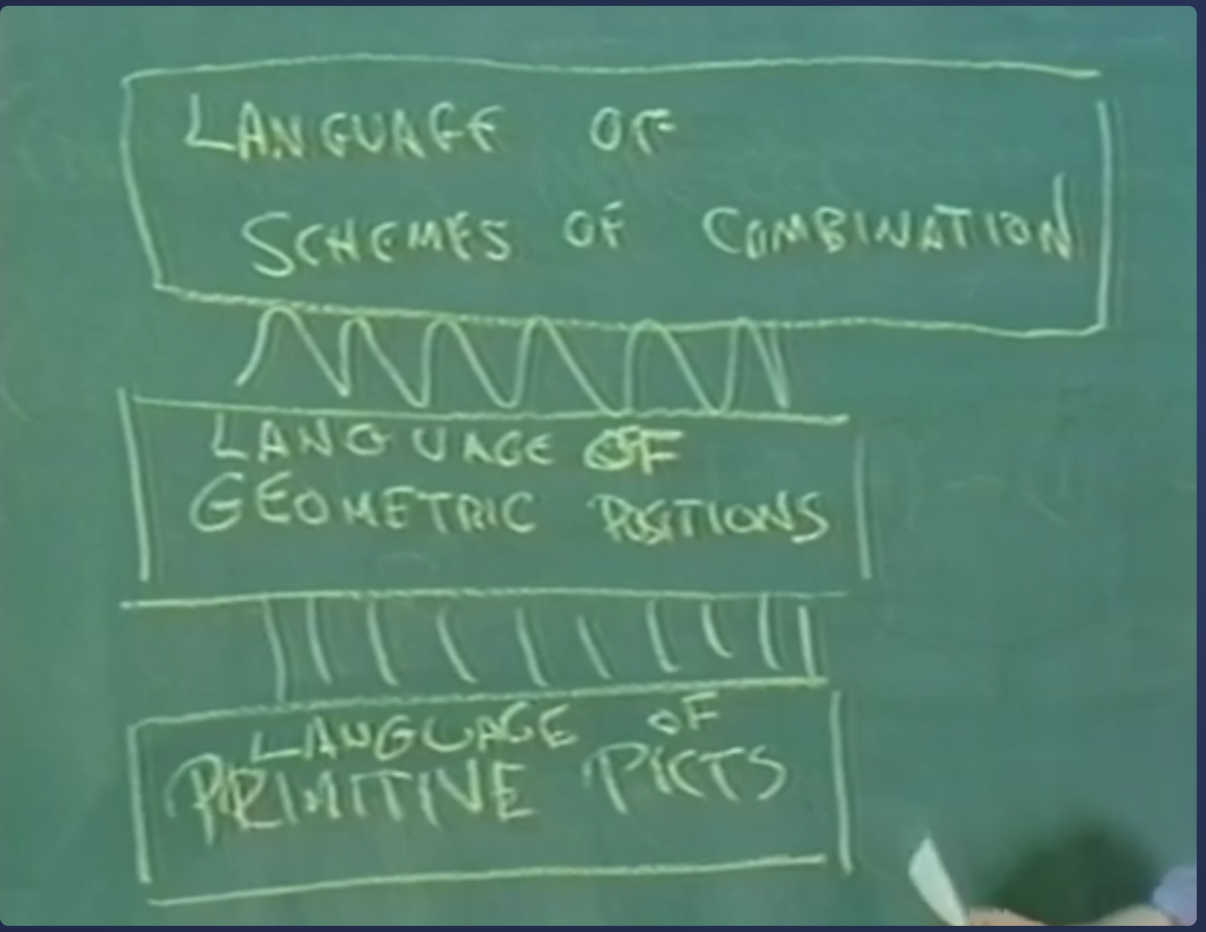
Lessons From Go's Net/HTTP Library for C++

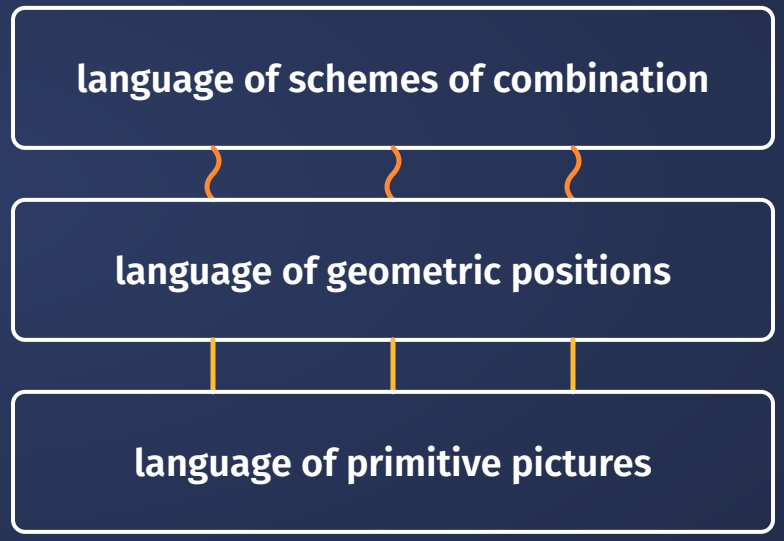
TIM VAN DEURZEN

Structure and Interpretation of Computer Programs



Layers of Language





Lecture 3A: Henderson's Escher Example

> … there is a more general point about the structure of the system as creating a language, viewing the engineering design process as one of creating language or rather one of creating a sort of sequence of layers of language.

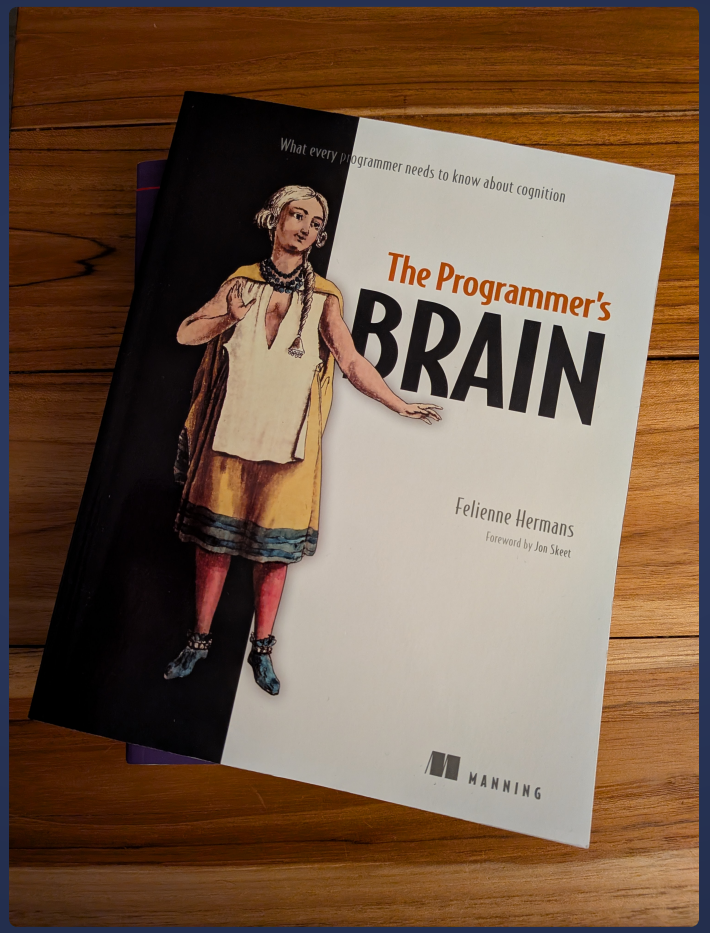
— Harold Abelson

> When you have levels of language, you're giving yourself different vocabularies for talking about design at different levels.

— Harold Abelson

Your Brain on Code

The Programmer's Brain



The Programmer's Brain

> "Programs must be written for people to read, and only incidentally for machines to execute."

— SICP

> "Programs must be written for people *and LLMs* to read, and only incidentally for machines to execute."

— This talk

Confusion: Lack of Knowledge

haskell

```haskell
adjacent :: [a] -> [(a, a)]
adjacent = zip <*> tail
```

lisp

```lisp
(defmacro unless* (test &body body)
  `(if ,test nil (progn ,@body)))
```

apl

```apl
2 2 2 2 2 ⊤ n
```

That's a rotate!

C++

```cpp
template<class It> It rotate(It first, It middle, It last) {
    if (first == middle) return last;
    if (middle == last) return first;
    It write = first;
    It next_read = first;
    for (It read = middle; read != last; ++write, ++read) {
        if (write == next_read) next_read = read;
        std::iter_swap(write, read);
    }
    rotate(write, next_read, last);
    return write;
}
```

Source: https://cppreference.com/cpp/algorithm/rotate

Confusion: Lack of Information

C++

```cpp
int main() {
    std::string key{"banana"};
    std::vector indices{0,3,5};

    auto result = extract_bytes(key, indices);
}
```

Chunking

C++

```cpp
std::vector vs{1, 1, 2, 3, 5};
//...
for (int i = 0; i < vs.size(); i++) {
    if (vs[i] == needle) {
        // ...
    }
}
// ...
```

Confusion: Lack of Processing power

Binary representation

basic

```basic
LET N2 = ABS (INT (N))
LET B$ = ""
FOR N1 = N2 TO 0 STEP 0
    LET N2 = INT (N1 / 2)
    LET B$ = STR$ (N1 - N2 * 2) + B$
    LET N1 = N2
NEXT N1
PRINT B$
RETURN
```

Lessons from Go's net/http Library

Porting from Go to C++23

Creating an HTTP server in Go

Go

```go
package main
import "net/http"

func find(w http.ResponseWriter, r *http.Request)      { w.Write([]byte("found")) }
func transform(w http.ResponseWriter, r *http.Request) { w.Write([]byte("ok")) }
func main() {
  mux := &http.ServeMux{}
  mux.HandleFunc("GET /std/find/{needle}", find)
  mux.HandleFunc("POST /std/ranges/transform", transform)

  server := http.Server{Handler: mux, Addr: ":8888"}
  server.ListenAndServe()
}
```

The HTTP server contract – Go

Go

```go
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}

// Any function with the right shape is a Handler.
type HandlerFunc func(ResponseWriter, *Request)

func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) {
    f(w, r)
}
```

> The bigger the interface, the weaker the abstraction.

— Go proverbs, Rob Pike

Middleware – Go

Go

```go
func WithLogger(logger *slog.Logger, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        logger.Info("Handling request ...")
        next.ServeHTTP(w, r)
    })
}
```

ServeMux – Go

Go

```go
type ServeMux struct { /* ... */ }

func (s ServeMux) ServeHTTP(ResponseWriter, *Request) { }

func (s ServeMux) Handle(pattern string, h Handler) { }

func (s ServeMux) HandleFunc(pattern string, h func(ResponseWriter, *Request)) { }
```

The HTTP server contract – C++

C++

```cpp
template <class H>
concept Handler =
   requires(H const& h, response_writer& w, request const& r) { h.serve_http(w, r); }
|| std::invocable<H const&, response_writer&, request const&>;

template <class H>
void invoke_handler(H const& h, response_writer& w, request const& r) {
    if constexpr (requires { h.serve_http(w, r); }) {
        h.serve_http(w, r);
    } else {
        std::invoke(h, w, r);
    }
}
```

Middleware – C++

C++

```
template<Handler H> auto with_logger(Logger logger, H next) {
    return [&](response_writer& w, request const & r) {
        logger.Log("Handling request ...");
        invoke_handler(next, w, r);
    };
}
```

Spot the bug

ServeMux - C++

C++

```
struct serve_mux {
  template <Handler H> void handle(std::string_view pattern, H handler) { /**/ }

  void serve_http(response_writer &w, request const &r) const { /**/ }
};
```

A Wrinkle in The Handler Language

C++

```
void login(response_writer &w, request const& r) {
    auto [user, pass] = credentials_from_request(r);

    if (not authenticated(user, pass)) {
        w.write_header(401);
        w.write("invalid user or password");
    }

    create_session(user);

    w.write_header(200);
    w.write("authenticated");
}
```

A Wrinkle in The Handler Language

C++

```
void login(response_writer &w, request const& r) {
    auto [user, pass] = credentials_from_request(r);

    if (not authenticated(user, pass)) {
        w.write_header(401);
        w.write("invalid user or password");
    } else {
        create_session(user);

        w.write_header(200);
        w.write("authenticated");
    }
}
```

A missing layer

???

The language of routing

`ServeMux, patterns`

The language of HTTP handlers

`Handler, ServeHTTP`

The Language of Testing

Tests for strconv

Shell

```
# Floating-point conversion test cases.
# Empty lines and lines beginning with # are ignored.
# The rest have four fields per line: type, format, input, and output.

# Table 1: Stress Inputs for Conversion to 53-bit Binary, < 1/2 ULP
float64 %b 5e+125 6653062250012735p+365
float64 %b 69e+267 4705683757438170p+841
float64 %b 999e-026 6798841691080350p-129
float64 %b 7861e-034 8975675289889240p-153
```

The Language of Testing - Table-Driven Tests

C++

```cpp
// ...

struct test_case {
  std::string_view in;
  std::vector<std::string_view> want;
};

auto tests = std::vector<test_case>{
    {.in = "/a/b/c", .want = std::vector{"a"sv, "b"sv, "c"sv}},
    {.in = "/a/b/", .want = std::vector{"a"sv, "b"sv, "/"sv}},
    {.in = "/", .want = std::vector{"/"sv}},
};

// ...

for (auto test : tests) {
  std::vector<std::string_view> got;
  auto rest = test.in;
  while (!rest.empty()) {
    auto [segment, remainder] = libhttp::detail::first_segment(rest);
```

Now Go design some languages!

API Design is Language Design

Lessons From Go's Net/HTTP Library for C++

TIM VAN DEURZEN

Resources

- The Programmer's Brain
- SICP Lecture recordings
- SICP Syllabus
- Go Testing By Example

Summary

- Design your APIs as layers of small languages
- Use familiar names and patterns
- Build your long-term memory by reading and reasoning about code